

第 35 讲

Batch、Epoch、Step

训练日志里的三个时间尺度到底怎样换算

《面向 LLM / Agent Systems 科研的深度学习零基础课》

本讲定位

第 32 讲已经学会 Dataset / DataLoader / Mini-Batch，第 34 讲又区分了 Full-Batch、SGD、Mini-Batch SGD，并引入 Gradient Accumulation。本讲专门解决一个很容易在论文、训练日志和开源代码中造成误读的问题：“训练了多少”到底用什么单位描述？我们将把 Batch、Epoch、micro-batch iteration、optimizer step、global step、gradient accumulation、global/effective batch、samples seen 和 tokens seen 统一到同一套计数体系中。

学习目标

学完本讲，你应该能够：

1. 看到 Dataset Size N 和 Batch Size B ，立即算出一个 Epoch 有多少个 Batch；
2. 正确解释 **Batch**、**Epoch**、**Iteration**、**Optimizer Step**、**Global Step**，并知道“Step”本身有歧义；
3. 理解 `drop_last=True/False` 会怎样改变一个 Epoch 的 Batch 数和实际样本访问量；
4. 在 Gradient Accumulation 存在时，区分 **micro-batch iteration** 与 **optimizer step**；
5. 计算单 GPU 和多 GPU 下的 global/effective batch size；
6. 对 LLM 训练进一步计算 **tokens per update** 与 **total tokens seen**；
7. 看懂论文中的“10k steps”“3 epochs”“batch size 256”“gradient accumulation 8”“warmup 2k steps”究竟指什么；
8. 判断两个训练实验在“训练预算”上是否真的可比；
9. 理解 Agent trajectory 数据中“1 个样本”“1 个序列”“1 个 step”的含义可能不同，避免把统计单位混在一起。

1 为什么 Batch、Epoch、Step 特别容易混？

原理

深度学习训练不是只有一层循环，而经常是**嵌套的时间尺度**：

$$\text{Epoch} \supset \text{Mini-Batch / Micro-Batch} \supset \text{Forward + Backward}$$

如果再加入 Gradient Accumulation、Distributed Data Parallel、Learning Rate Scheduler、Checkpoint、Validation，那么“step”这个词会被不同代码用于不同层级。

最简单的训练代码通常长这样：

```
for epoch in range(num_epochs):
    for x, y in dataloader:
        optimizer.zero_grad()
        pred = model(x)
        loss = loss_fn(pred, y)
        loss.backward()
        optimizer.step()
```

如果没有 Gradient Accumulation，那么在这个最简单版本里：

$$1 \text{ Batch} \approx 1 \text{ Forward/Backward Iteration} \approx 1 \text{ Optimizer Step}$$

正因为最开始它们常常“一对一”，很多初学者就把 Batch、Iteration、Step 混成一个词。可一旦加入 Gradient Accumulation，这个一对一关系立刻失效。

常见误区

以后看到日志中的 `step=1000`，不要马上理解成“看了 1000 个 Batch”或者“参数更新了 1000 次”。第一反应应该是：**这里的 step 到底是 micro-batch iteration、optimizer step、scheduler step，还是作者自己定义的 global step？**

2 先把最基本的三个对象固定下来

2.1 Batch：一次送进模型的一组样本

假设 Dataset 有 N 个样本，DataLoader 设置：

$$B = \text{batch size}$$

那么一个 Batch 通常包含 B 个样本。

如果监督学习输入为：

$$X : (B, D)$$

那么 B 就是 Batch Dimension。对于 LLM，常见输入是：

$$\text{input_ids} : (B, T)$$

Embedding 以后变成：

$$X : (B, T, D)$$

这里 B 仍然表示“这一批有多少条序列”， T 是序列长度， D 是隐藏维度。

具体数字例子

假设有 1000 条训练样本，`batch_size=128`。那么模型每次最多同时处理 128 条样本。前 7 个完整 Batch 会处理 $7 \times 128 = 896$ 条，还剩 104 条，因此如果不丢弃最后一批，一个 Epoch 一共会有 8 个 Batch。

2.2 Epoch：训练集被“完整遍历一次”的时间尺度

在最常见的 `DataLoader(shuffle=True)` 且使用默认不放回采样时，一个 Epoch 可以理解为：

让训练集中每个样本都有一次被 `DataLoader` 取出的机会。

这不等于“模型学会了所有样本”，也不等于“每个参数只更新一次”。一个 Epoch 里可能包含很多个 Parameter Update。

Epoch 的严格边界

“一个 Epoch = 每个样本恰好出现一次”只适用于常见的有限 Dataset + 默认不放回采样语境。如果使用 weighted sampling、replacement sampling、streaming dataset、无限数据流或者动态生成数据，Epoch 可能只是人为定义的一段训练长度，甚至根本没有自然意义。

2.3 Step：必须先问“谁的 Step?”

在深度学习工程中，“Step”至少可能指下面几件不同的事：

术语	更准确的含义
micro-batch iteration	取出一个 micro-batch，完成一次 forward/backward
optimizer step	真正执行一次 <code>optimizer.step()</code> ，参数发生一次更新
global step	某个框架维护的全局计数器，通常但不总是等于 optimizer step
scheduler step	学习率调度器推进一次
evaluation step	训练到某个计数点触发一次验证
logging step	训练到某个计数点记录一次日志

最低闭环能力

以后你读论文或代码时，如果看到“trained for 50k steps”，必须继续寻找：

- 这个 step 是 optimizer update 还是 batch iteration?
- 有没有 gradient accumulation?
- 是单 GPU 还是多 GPU?
- 一个 step 里实际包含多少 sample / sequence / token?

否则 “50k steps” 本身几乎无法告诉你训练预算到底有多大。

3 没有 Gradient Accumulation 时, Batch、Epoch、Step 怎样换算?

设:

- Dataset Size 为 N ;
- Batch Size 为 B ;
- 训练 Epoch 数为 E ;
- 每个 Batch 都立即做一次 `optimizer.step()`。

如果:

`drop_last=False`

那么每个 Epoch 的 Batch 数为:

$$M = \left\lceil \frac{N}{B} \right\rceil$$

如果:

`drop_last=True`

那么:

$$M = \left\lfloor \frac{N}{B} \right\rfloor$$

在没有 Gradient Accumulation 时:

$$\text{optimizer steps per epoch} = M$$

总 optimizer steps:

$$S_{total} = E \times M$$

具体数字例子

$N = 10,000$, $B = 100$, $E = 5$, 且可以整除。

$$M = 10,000/100 = 100$$

所以:

$$S_{total} = 5 \times 100 = 500$$

模型一共经历 5 个 Epoch、500 个 Batch、500 次 optimizer update。

4 为什么 `drop_last` 会改变计数？

实验代码中设置：

$$N = 23, \quad B = 6$$

当 `drop_last=False`：

[6, 6, 6, 5]

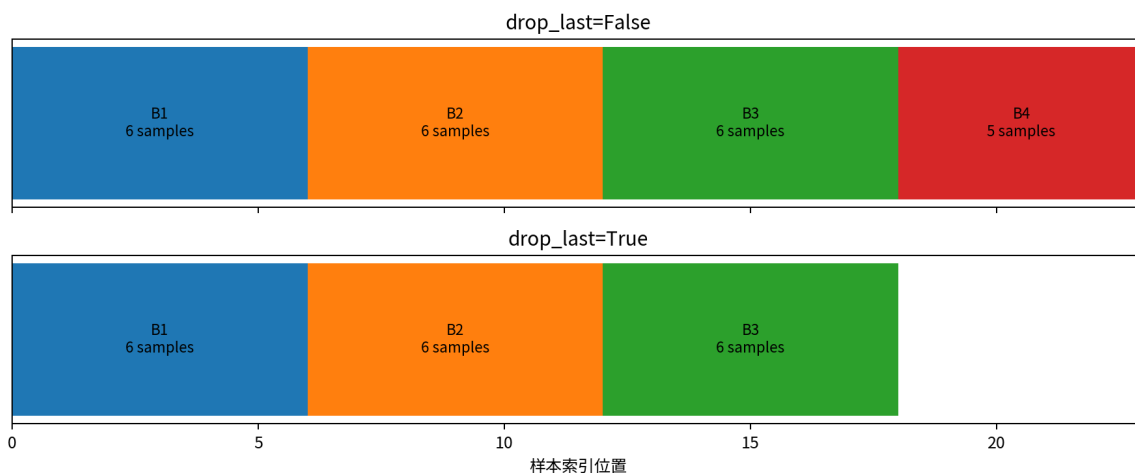
一共 4 个 Batch。

当 `drop_last=True`：

[6, 6, 6]

一共 3 个 Batch，最后 5 个样本被丢弃。

N=23, batch_size=6：一个 Epoch 怎样被切成 Batch



原理

`drop_last=True` 不只是“最后一批少一点”这么简单，它同时改变：

- 每个 Epoch 实际看到的样本数；
- 每个 Epoch 的 Batch 数；
- 如果每 Batch 更新一次，则改变 optimizer step 数；
- 某些依赖固定 Batch 统计量的方法可能因此行为不同。

与科研的关系

比较两个实验时，如果一个实验使用 `drop_last=True`，另一个为 `False`，Dataset 又很小，那么“相同 Epoch”未必意味着相同样本访问次数。这类差异在小样本实验、Agent trajectory 数据集、医学数据集等场景尤其值得留意。

5 加入 Gradient Accumulation 后，最重要的关系改变了

第 34 讲已经学过：Gradient Accumulation 的思想是把多个 micro-batch 的梯度先累起来，然后再更新一次参数。

假设：

$$A = \text{gradient accumulation steps}$$

那么代码逻辑更接近：

```
optimizer.zero_grad()
for i, batch in enumerate(loader):
    loss = forward_and_loss(batch)
    (loss / A).backward()

    if accumulated_A_micro_batches:
        optimizer.step()
        optimizer.zero_grad()
```

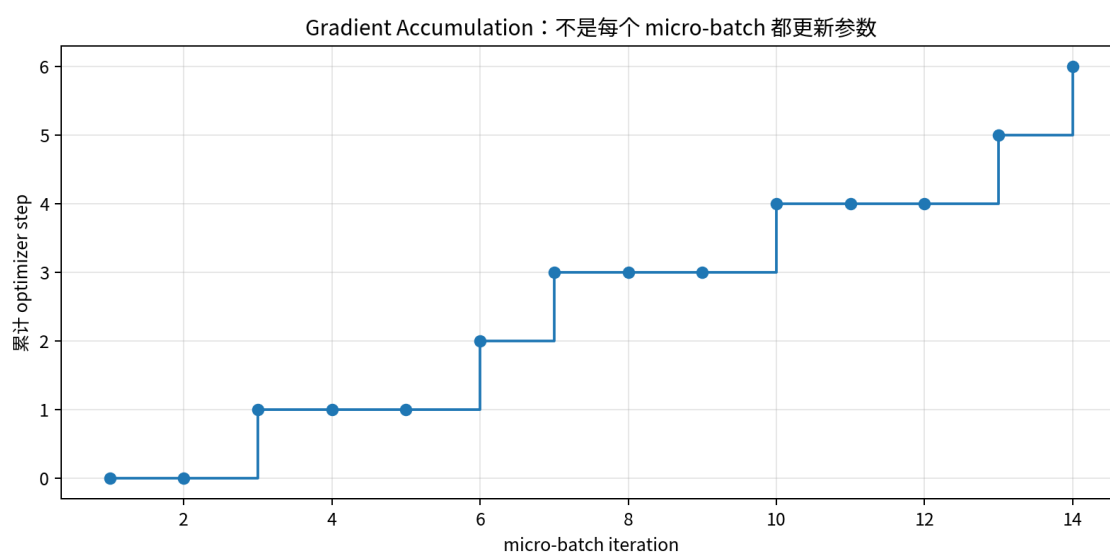
此时：

$$\text{micro-batch iteration} \neq \text{optimizer step}$$

例如一个 Epoch 有 7 个 micro-batch， $A = 3$ 。如果 Epoch 末尾把剩余梯度也更新一次，那么：

$$\text{optimizer steps per epoch} = \left\lceil \frac{7}{3} \right\rceil = 3$$

并不是 7。



图中每经过一个 micro-batch，forward/backward 都发生一次；但只有积累到指定数量，或者到达 Epoch 末尾需要 flush 时，optimizer step 才增加。

常见误区

如果你的训练代码使用 Gradient Accumulation, 却把“每个 dataloader batch”都叫作 global step, 那么日志里的 step 数就可能比真正的 parameter update 数大 A 倍。读别人的训练日志时, 一定要看实现。

6 一个更细的边界问题: Epoch 结束时, 剩余梯度怎么办?

假设每个 Epoch 有:

$$M = 7$$

个 micro-batch, 累积:

$$A = 3$$

个 micro-batch 更新一次。

前 6 个 micro-batch 很容易分成两组:

$$[1, 2, 3], \quad [4, 5, 6]$$

最后只剩:

$$[7]$$

这时有至少三种实现策略:

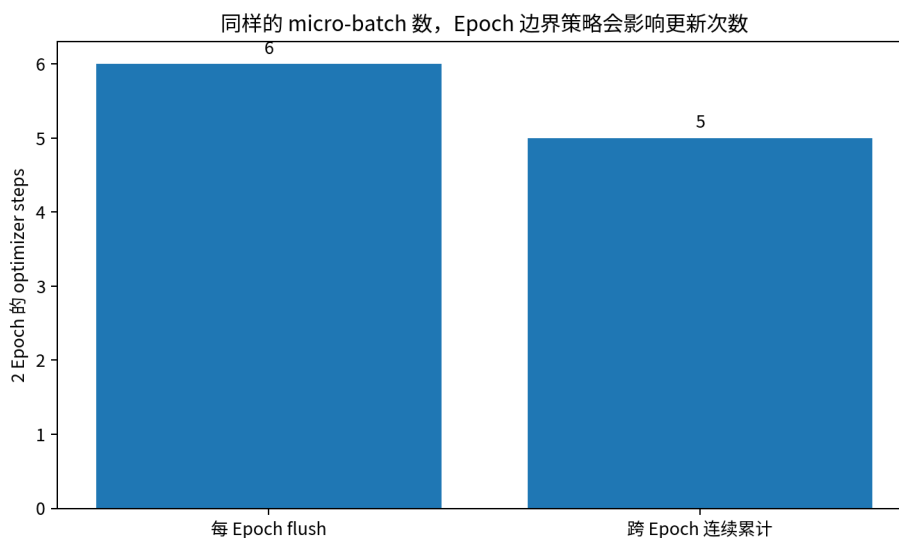
1. **Epoch 末立即 flush:** 用剩余 1 个 micro-batch 的梯度做一次更新;
2. **跨 Epoch 继续累计:** 把下一个 Epoch 的前两个 micro-batch 与它组成 3 个再更新;
3. **错误地直接丢弃:** 剩余梯度白算, 不做更新。

在训练 2 个 Epoch 时:

$$\text{每 Epoch flush} = 2 \times \left\lceil \frac{7}{3} \right\rceil = 6 \text{ steps}$$

而跨 Epoch 连续累计:

$$\left\lceil \frac{14}{3} \right\rceil = 5 \text{ steps}$$



与科研的关系

这个差异通常不是论文的核心创新，但它会影响复现实验时的 update 次数、最后一组 effective batch、scheduler 进度和训练预算。真正复现实验时，要以代码实现为准，而不能只靠配置文件里的“epochs=2, accumulation=3”自行猜测。

7 Effective Batch Size：一次 Parameter Update 究竟汇总了多少样本？

单 GPU，micro-batch size 为 B_{micro} ，Gradient Accumulation 为 A ：

$$B_{effective} = B_{micro} \times A$$

如果有 G 张 GPU，每张 GPU 每次处理 B_{micro} 个样本，并使用数据并行同步梯度：

$$B_{global} = B_{micro} \times G \times A$$

具体数字例子

每张 GPU 一次处理 2 条序列，共 4 张 GPU，Gradient Accumulation 为 8：

$$B_{global} = 2 \times 4 \times 8 = 64$$

也就是每次 optimizer update 汇总 64 条序列的梯度信息。

“batch size”在论文里也可能有歧义

作者写“batch size=64”时，可能指：

- 每张 GPU 的 micro-batch；
- 所有 GPU 合计的 global batch；
- 加上 gradient accumulation 后的 effective batch；

- 甚至是 token batch, 而不是 sequence batch。

所以看论文时要结合 hardware、accumulation、world size 一起读。

8 LLM 训练为什么更喜欢用 Tokens 来描述预算？

对固定长度序列，假设：

- global effective batch = B_g ;
- sequence length = T ;

那么一次 optimizer update 对应的 token slot 约为：

$$\text{token slots per update} = B_g \times T$$

例如：

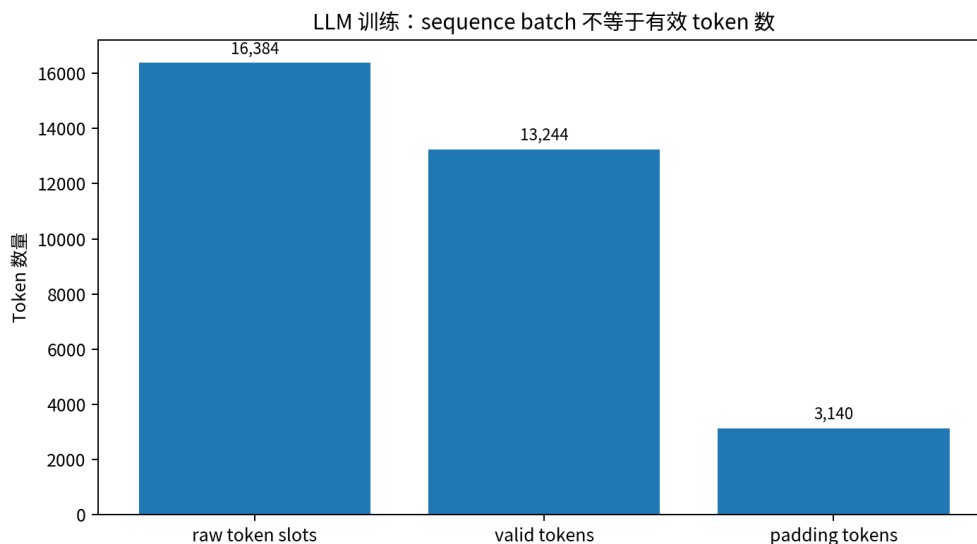
$$B_g = 64, \quad T = 2048$$

则：

$$64 \times 2048 = 131,072$$

个 token slots/update。

但是如果有 Padding, 真正有效 Token 可能更少。



本讲实验里，8 条长度上限为 2048 的序列共有：

$$16,384 \text{ raw token slots}$$

但有效 token 只有：

$$13,244$$

Padding 占：

3,140

原理

因此在 LLM 训练里，**total tokens seen** 往往比 Epoch 更有跨实验可比性。尤其当数据来自 streaming dataset、去重语料混合、动态 packing 或不同 sequence length 时，“训练了 1 Epoch”未必是一个稳定、可比较的预算单位。

9 Global Step 与 Scheduler：为什么计数错一个层级，Learning Rate 就会跑偏？

Gradient Accumulation 下，一个常见工程错误是：每个 micro-batch 都调用一次 learning-rate scheduler，而 optimizer 实际每 A 个 micro-batch 才更新一次。

实验设置：

20 micro-batches, $A = 4$

所以真正 optimizer steps 只有：

$$20/4 = 5$$

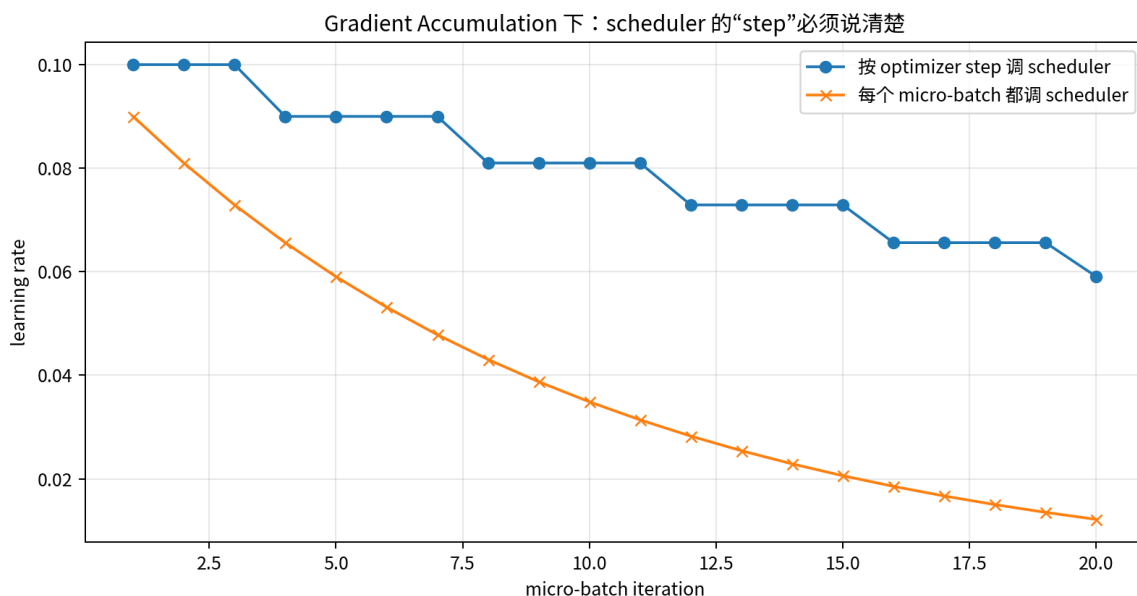
如果每次 scheduler step 都把学习率乘 0.9：

正确按 optimizer step：

$$0.1 \times 0.9^5 = 0.059049$$

错误按 micro-batch：

$$0.1 \times 0.9^{20} \approx 0.012158$$



常见误区

同样的“warmup 1000 steps”也必须问：是 1000 个 optimizer updates，还是 1000 个 micro-batches？在 accumulation=8 时，两者代表的实际训练量可能相差约 8 倍。

10 把 PyTorch 训练代码逐行映射到这些计数器

考虑下面这段带 accumulation 的简化代码：

```
optimizer.zero_grad()
global_step = 0

for epoch in range(E):
    for micro_step, (x, y) in enumerate(loader):
        pred = model(x)
        loss = loss_fn(pred, y) / A
        loss.backward()

        if (micro_step + 1) % A == 0:
            optimizer.step()
            optimizer.zero_grad()
            global_step += 1
```

逐项解释：

- epoch：外层完整遍历次数；
- micro_step：当前 Epoch 内第几个 DataLoader Batch；
- 每执行一次 loss.backward()：完成一次 micro-batch 的梯度贡献；
- 每执行一次 optimizer.step()：参数真正更新一次；
- 如果代码把 global_step += 1 放在 optimizer.step 后，那么 global step 就是 optimizer step；
- 如果把 global step 放到内层每个 batch 都加 1，那么它就变成了 micro-batch iteration。

读代码时最有效的方法

不要只看变量名叫不叫 step。看计数器在什么位置加 1。如果它紧跟在 optimizer.step() 后面，通常代表 parameter update；如果每个 DataLoader Batch 都加 1，通常代表 batch iteration。

11 一个完整数字例子：把所有时间尺度统一起来

设：

$$N = 1000, \quad B = 128, \quad E = 10, \quad A = 4$$

且：

drop_last=False

每个 Epoch 的 micro-batch 数:

$$M = \left\lceil \frac{1000}{128} \right\rceil = 8$$

如果每个 Epoch 末尾 flush:

$$S = \left\lceil \frac{8}{4} \right\rceil = 2 \text{ optimizer steps/epoch}$$

训练 10 Epoch:

80 micro-batch iterations

20 optimizer steps

样本访问次数:

$$10 \times 1000 = 10,000$$

如果每条序列固定 512 tokens:

$$10,000 \times 512 = 5,120,000$$

raw token slots。

注意：这里的“20 optimizer steps”很少，是因为一次 update 汇总了 4 个 micro-batch。这个例子也说明，同样 10 Epoch，不同 accumulation 会导致 optimizer update 次数完全不同。

12 科研比较：为什么“相同 Epoch”经常不是公平预算？

假设两个实验 Dataset 相同，都训练 10 Epoch:

实验	Batch Size	Accumulation	Optimizer Steps / Epoch
A	32	1	多
B	256	1	少
C	32	8	更少

即使总样本访问次数相同，它们也可能有不同的：

- parameter update 次数；
- gradient noise；
- learning-rate schedule 进度；
- wall-clock；
- communication 开销；
- optimizer state 更新次数。

因此比较实验至少要明确自己控制的是哪一种预算：

1. 相同 Epoch;
2. 相同 samples seen;
3. 相同 tokens seen;
4. 相同 optimizer steps;
5. 相同 FLOPs / compute;
6. 相同 wall-clock;
7. 相同 API cost 或 dollar budget。

与科研的关系

对你的 Agent Systems 研究，“相同预算”以后经常不只是 GPU 训练时间。还可能是：相同 LLM API calls、相同 input/output tokens、相同 trajectory 数、相同 tool calls、相同 environment interactions。把“预算单位”定义清楚，是可复现实验设计的一部分。

13 LLM 论文里常见的几个训练配置怎么读？

假设论文写：

```
micro_batch_size_per_gpu = 2
num_gpus = 8
gradient_accumulation_steps = 16
sequence_length = 4096
training_steps = 10000
```

如果 training steps 指 optimizer updates，那么：

$$B_{global} = 2 \times 8 \times 16 = 256 \text{ sequences/update}$$

固定满长度时：

$$256 \times 4096 = 1,048,576 \text{ token slots/update}$$

10,000 optimizer steps 对应：

$$\approx 10.49 \text{ billion token slots}$$

但仍然要继续确认：

- 是否有 padding / packing;
- 是否把 label mask 掉了一部分;
- training steps 是否真的指 optimizer step;
- 多 GPU 下 batch size 是否已经是 global batch;
- 数据是否重复多轮使用。

14 Agent Systems 中，“Step”更容易产生第二层歧义

Agent 研究里还有一个额外问题：**environment step / agent step** 与 **optimizer step** 完全不是一回事。

例如一个 trajectory:

$$\tau = (s_1, a_1, o_1, s_2, a_2, o_2, \dots)$$

可能包含 30 次 Agent-Environment interaction。

那么至少存在:

- 30 个 environment steps;
- 1 条 trajectory;
- 若将 trajectory 作为训练样本, 可能只有 1 个 sequence sample;
- 训练时又可能经过若干 micro-batch iterations;
- 最终只触发若干 optimizer steps。

Agent 数据中的相关性

同一条 trajectory 内相邻 transition 高度相关。把 100 条 trajectory 拆成 3000 个 transition 后, 不能简单把“3000”理解为 3000 个独立样本。做 Train/Validation/Test Split 时, 很多场景更适合按 trajectory / task / environment 实例切分, 而不是把同一轨迹的片段随机散到不同集合里。

15 训练日志里最常见的 8 个误读

1. 把 **Batch** 等同于 **Step**: 有 accumulation 时通常不成立;
2. 把 **Epoch** 等同于固定更新次数: Batch Size 改变后 step/epoch 会变;
3. 忽略 **drop_last**: 小 Dataset 时样本访问量会改变;
4. 把 **global batch** 当作 **per-device batch**: 多 GPU 下可能差一个 world size;
5. 把 **sequence batch** 当作 **token batch**: 不同长度序列实际 token 数不同;
6. **scheduler** 跟错 **step**: accumulation 下学习率计划可能提前走完;
7. 只报 **epochs**, 不报 **steps/tokens**: 复现者无法判断真实预算;
8. 把 **Agent environment step** 当 **optimizer step**: 两个“step”属于完全不同层级。

16 读论文 Training Setup 时的检查清单

以后看到训练配置, 可以按下面顺序检查:

1. Dataset 有多大? N 是 sample、sequence、trajectory 还是 token?
2. Batch Size 是 per-device、global, 还是 effective?

3. 有多少 device?
4. Gradient Accumulation 是多少?
5. 每次 optimizer update 汇总多少 sample / sequence / token?
6. 训练用 Epoch 还是 total steps 终止?
7. “step” 具体指什么?
8. Scheduler / warmup 是按哪个 step 计数?
9. drop_last / packing / padding / masking 怎样设置?
10. Validation / checkpoint / logging 的 cadence 依据哪个计数器?

一句话总结

Epoch 描述“数据被遍历了多少轮”；**Batch / micro-batch** 描述“每次计算吃多少数据”；**optimizer step** 描述“参数真正更新了多少次”；**tokens seen** 描述“模型实际处理了多少 token”。Gradient Accumulation 和分布式训练会让这些尺度彼此不再一一对应。

17 与后续课程的关系

下一讲将专门学习 **Learning Rate**。你已经能够看见为什么 Learning Rate 不能脱离 Step 定义单独讨论：

$$\theta_{t+1} = \theta_t - \eta_t \nabla L_t$$

这里的 t 通常表示一次 optimizer update。因此当 scheduler 根据 t 改变 η_t 时，如果你把 micro-batch iteration 错当作 optimizer step，整个 learning-rate schedule 的时间轴都会改变。

对 LLM 训练也是一样：Warmup、Cosine Decay、Checkpoint、Evaluation 都需要依附在某个明确的训练计数器上。第 36 讲将从这里继续。